

LAG

Keep missing values and order meaningful

01 / THE SAS EXAMPLE

```
data test;
  input id value;
  prev_value = lag(value);
  prev2_value = lag2(value);
  change = value - prev_value;
  datalines;
1 10
2 15
3 12
4 18
5 20
;
run;
proc print data=test noobs;
run;
```

REPORTED TEST SETUP / 21 SEP 2026

Claude Sonnet 4.5 / AWS Bedrock GovCloud
Claude Code 2.1.278 / effortLevel=high / prompt v0.1.0

ATTACH SOURCE OR PASTE IT AFTER THE PROMPT. START FRESH EACH TIME.

WHAT YOU WILL PRACTICE

State what the model should preserve when there is no previous value.

Each LAG call executes on every row in this program. The supplied order defines the previous values. Empty starting positions mean no earlier observation; they remain missing.

EXPECTED RESULT

id	value	prev_value	prev2_value	change
1	10	.	.	.
2	15	10	.	5
3	12	15	10	-3
4	18	12	15	6
5	20	18	12	2

A period denotes SAS numeric missing; pandas may display NaN or <NA>. Compare missing positions as well as numbers. Conditional LAG calls require a separate analysis.

Reported output matches: Prompt A 3/3 | Prompt B 3/3

Reference checked in Python; SAS execution pending.

Prompt A: leave choices open

High degrees of freedom / Notice the decisions the model makes for you.

COPY THIS PROMPT + THE SAS EXAMPLE

Convert this SAS code to Python.
Show code in this response. Do not create or run files, or read existing Python files.

REPORTED TRIAL RESULTS / 21 SEP 2026

All A runs chose row shifts

Run	Method	Output match
A1	shift(1) + shift(2)	Yes
A2	shift(1) + shift(2)	Yes
A3	shift(1) + shift(2)	Yes

All three report matching values and missing positions.
No A run reportedly included assertions.
Shared method does not establish identical full responses.

SOURCES: results-20260921.csv; trial-log-20260921.md; evidence-review-20260922.md

WHY CHOICES REMAIN OPEN

- 1

A short request can succeed
All A reports describe shift(1) and shift(2). The source calls both SAS functions on every row.
- 2

Check meaning as well as numbers
The first prior value is unavailable. Its missing value must remain missing after subtraction.
- 3

Keep the sequence intact
Sorting, dropping rows or wrapping the final value into the first row changes the calculation.

ANTI-PATTERN: "FILL THE BLANKS"

Replacing the first missing prior value with zero manufactures a change of 10. Missing means there was no earlier observation.

Preserve missing positions and the five-row order.

Prompt B: make the choices explicit

Low degrees of freedom / State the behavior, then ask for a check.

COPY THIS PROMPT + THE SAME SAS EXAMPLE

Convert this SAS code to Python using pandas.
Show code in this response. Do not create or run files, or read existing Python files.

- Recreate id and value, preserving all five rows and their order.
- Each SAS LAG call executes on every row in this program.
- prev_value: use value.shift(1), with no frequency argument.
- prev2_value: use value.shift(2), with no frequency argument.
- change: subtract prev_value from value.
- Leave unavailable prior values and resulting changes missing. Do not fill them with zero, drop rows, sort, or wrap values.
- Include assertions for the original id sequence and these lists:
prev_value: [missing, 10, 15, 12, 18]
prev2_value: [missing, missing, 10, 15, 12]
change: [missing, 5, -3, 6, 2]
Use pd.isna to test missing positions.
- Return one complete script. Compute results from value; do not hard-code the result columns.

Briefly explain why the first change is missing.

WHY THESE DETAILS HELP

- 1

Protect the original sequence
Use row shifts without a frequency argument. Preserve all five ids and their order.
- 2

Limit the equivalence
Shift fits these per-row calls. Conditional SAS LAG calls advance their queue only when executed.
- 3

Make missing values checkable
B1-B3 report assertions and an explanation of the first missing change. Use pd.isna for missing positions.

REPORTED RESULTS / THREE B RUNS

Run	Output match	Assertions
B1	Yes	Passed
B2	Yes	Passed
B3	Yes	Passed

CHECK IT YOURSELF
Row 3 change = -3; the first change stays missing.
Compare every value, missing position and original id.

One behavior, three implementations

Reproduce earlier-row values only where each SAS LAG occurrence executes on every row. Preserve missing history and the original sequence.

SAS / SOURCE BEHAVIOR

```
%macro transform(in=, out=);
data &out;
  set &in;
  prev_value = lag(value);
  prev2_value = lag2(value);
  change = value - prev_value;
run;
%mend;
```

Macro transform supplies source/result names. Each LAG occurrence maintains a separate queue initialized with missing values.

PYTHON / PANDAS

```
out = df.copy(deep=True)
out['prev_value'] = out['value'].shift(1)
out['prev2_value'] = out['value'].shift(2)
out['change'] = (
    out['value'] - out['prev_value'])
```

Series.shift with no frequency moves values by row. Out is a copy; arithmetic preserves unavailable history as missing.

R / BASE

```
lag_rows <- function(x, k) {
  n <- length(x)
  if (n == 0L) return(numeric(0))
  k <- min(as.integer(k), n)
  c(rep(NA_real_, k), head(x, n - k))
}
out <- df
out$prev_value <- lag_rows(out$value, 1L)
out$prev2_value <- lag_rows(out$value, 2L)
out$change <- out$value - out$prev_value
```

lag_rows pads with NA and clamps the lag for short vectors. stats::lag changes time-series attributes and is not this row-shift operation.

Missing means no prior observation

Do not fill the first prior value with zero: that manufactures a change of 10. Keep the two missing starting positions of the two-row lag.

Respect call history

Conditional SAS LAG calls advance their queue only when executed. Such a program requires separate analysis; unconditional shifts are insufficient.

SAS LAG queue semantics	pandas.Series.shift	R lag (time series)
-------------------------	---------------------	---------------------

Three checks that earn the pass

Use the same inputs and expected behavior in each language. Check the contract as well as the values.

T1 / Original known-answer fixture

INPUT / CONDITION

Values 10,15,12,18,20 in id order 1-5.

EXPECTED

lag1=[NA,10,15,12,18]; lag2=[NA,NA,10,15,12];
change=[NA,5,-3,6,2].

WHAT THIS CATCHES

Wrong lag direction, wraparound, missing-to-zero fills or reordered predecessors.

T2 / Added one-row data set

INPUT / CONDITION

One row: id=41, value=7.

EXPECTED

Both previous-value columns and change are missing; one row remains.

WHAT THIS CATCHES

Lag padding longer than input, zero filling, wraparound and invalid short-vector indexing.

T3 / Preserve order, types and missing

INPUT / CONDITION

Original rows with non-default Python/R labels; nullable Python numeric values.

EXPECTED

Input unchanged; id/value order preserved; numeric results and exact missing masks.

WHAT THIS CATCHES

Input mutation, row sorting/dropping, text outputs or missing-position drift.

REFERENCE CODE + TESTS: [module-10-lag/appendix/](#)

Run: `python tools/run_appendix_tests.py --modules 10`

New reference code, separate from historical trials. SAS checks await execution; scope is stated in the companion README.